PyTorch CIFAR-10 图像分类项目报告

一、项目概述

本次考核提供两个项目选项，第一个项目对 GPU 算力有较高要求，需要独立显卡支持。考虑到本人电脑仅配备集成显卡，无独立 GPU，因此选择第二个基于 CPU 即可完成的图像分类项目，并在其基础上完成 ResNet 模型的训练与外部图片测试。

本次项目复现了 GitHub 上的 pytorch-cifar 项目，在 CIFAR-10 数据集上训练图像分类模型。CIFAR-10 包含 10 个类别（飞机、汽车、鸟、猫、鹿、狗、青蛙、马、船、卡车），共 50000 张训练图片和 10000 张测试图片，每张图片大小为 32×32 像素。

考核要求：

1. 选取 1-2 个 ResNet 模型架构进行训练和推理
2. 自行选取数据集外的图片，对训练好的模型进行额外测试

二、实验环境

1. 操作系统：Windows 11
2. Python 版本：311
3. PyTorch 版本：2.4.1
4. 硬件：CPU（无 GPU 加速）
5. 环境管理：Conda 虚拟环境

使用 Conda 创建独立环境并安装依赖，避免污染系统 Python：

```
(pytorch-cifar) C:\Users\chuxi>conda activate pytorch-cifar

(pytorch-cifar) C:\Users\chuxi>pip install torch torchvision
Collecting torch
  Using cached torch-2.4.1-cp38-cp38-win_amd64.whl.metadata (27 kB)
Collecting torchvision
  Using cached torchvision-0.19.1-cp38-cp38-win_amd64.whl.metadata (6.1 kB)
Collecting filelock (from torch)
  Using cached filelock-3.16.1-py3-none-any.whl.metadata (2.9 kB)
Collecting typing-extensions>=4.8.0 (from torch)
  Using cached typing_extensions-4.13.2-py3-none-any.whl.metadata (3.0 kB)
Collecting sympy (from torch)
  Using cached sympy-1.13.3-py3-none-any.whl.metadata (12 kB)
Collecting networkx (from torch)
  Using cached networkx-3.1-py3-none-any.whl.metadata (5.3 kB)
Collecting jinja2 (from torch)
  Using cached jinja2-3.1.6-py3-none-any.whl.metadata (2.9 kB)
Collecting fsspec (from torch)
  Using cached fsspec-2025.3.0-py3-none-any.whl.metadata (11 kB)
Collecting numpy (from torchvision)
  Using cached numpy-1.24.4-cp38-cp38-win_amd64.whl.metadata (5.6 kB)
Collecting pillow!=8.3.*,>=5.3.0 (from torchvision)
  Using cached pillow-10.4.0-cp38-cp38-win_amd64.whl.metadata (9.3 kB)
Collecting MarkupSafe>=2.0 (from jinja2->torch)
  Using cached MarkupSafe-2.1.5-cp38-cp38-win_amd64.whl.metadata (3.1 kB)
Collecting mpmath<1.4,>=1.1.0 (from sympy->torch)
  Using cached mpmath-1.3.0-py3-none-any.whl.metadata (8.6 kB)
Downloading torch-2.4.1-cp38-cp38-win_amd64.whl (199.4 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━ 199.4/199.4 MB 27.4 MB/s eta 0:00:00
Downloading torchvision-0.19.1-cp38-cp38-win_amd64.whl (1.3 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━ 1.3/1.3 MB 33.1 MB/s eta 0:00:00
Downloading pillow-10.4.0-cp38-cp38-win_amd64.whl (2.6 MB)
   ━━━━━━━━━━━━━━━━━━━━━━━━ 2.6/2.6 MB 36.4 MB/s eta 0:00:00
Downloading MarkupSafe-2.1.5-cp38-cp38-win_amd64.whl (17 kB)
Downloading mpmath-1.3.0-py3-none-any.whl (536 kB)
   ━━━━━━━━━━━━━━━━━━━━━━━━ 536.2/536.2 kB 17.1 MB/s eta 0:00:00
Installing collected packages: mpmath, typing-extensions, sympy, pillow, numpy, networkx, MarkupSafe, fsspec, filelock, jinja2, tor
ch, torchvision
Successfully installed MarkupSafe-2.1.5 filelock-3.16.1 fsspec-2025.3.0 jinja2-3.1.6 mpmath-1.3.0 networkx-3.1 numpy-1.24.4 pillow-
10.4.0 sympy-1.13.3 torch-2.4.1 torchvision-0.19.1 typing-extensions-4.13.2
```

三、复现过程

3.1 模型一：SimpleDLA

3.1.1 模型简介

SimpleDLA 是一种轻量级的图像分类网络，参数量较小，适合快速验证流程。

3.1.2 训练过程

（1）训练轮数：100 轮

（2）训练耗时：约 20 小时

（3）模型保存：准确率创新高时自动保存

（4）遇到的问题及解决：

① 项目原代码中的进度条函数 progress_bar 使用了 Linux 系统的 stty size 命令获取终端宽度，在 Windows 环境下执行会报错 'stty' 不是内部或外部命令，导致训练中断。

```
(pytorch-cifar) C:\Users\chuxi\Desktop\cifar10-project\pytorch-cifar-master>python main.py
'stty' 不是内部或外部命令，也不是可运行的程序
或批处理文件。
Traceback (most recent call last):
  File "main.py", line 15, in <module>
    from utils import progress_bar
  File "C:\Users\chuxi\Desktop\cifar10-project\pytorch-cifar-master\utils.py", line 45, in <module>
    _, term_width = os.popen('stty size', 'r').read().split()
```

解决方案：修改 utils.py 文件，重写 progress_bar 函数，移除对 stty 的依赖，改为每 10 个 batch 打印一次当前进度，不再使用行内刷新和宽度计算。

修改后的 progress_bar 函数：

```
def progress_bar(batch_idx, total_batches, msg):
    """Simple progress bar that works on Windows"""
    if batch_idx % 10 == 0 or batch_idx == total_batches - 1:
        print(f'[{batch_idx+1:3d}/{total_batches:3d}] {msg}')
```

② 程序首次运行时需自动下载 CIFAR-10 数据集，但默认连接到国外服务器，下载速度极慢。

解决方案：使用国内清华镜像站手动下载数据集压缩包 cifar-10-python.tar.gz，并将其放入项目根目录下的 ./data/ 文件夹中，程序自动识别并跳过下载步骤。

③ 长时间训练可能电脑休眠、断电或手动停止而中断，若不支持断点续训，则需要从头开始，浪费大量时间。

解决方案：项目本身支持 --resume 参数，可从中断处恢复训练，在启动时使用语句：python main.py --resume，程序会自动加载 ./checkpoint/ckpt.pth 中保存的最新模型权重、训练轮次和最佳准确率，从中断处继续训练。

3.1.3 测试结果

以下是在训练过程中截取的关键阶段日志，展示了模型准确率 Acc 随训练轮数 Epoch 逐步上升过程：

训练初期（第 1-2 轮）：模型刚开始学习，第 1 轮训练集准确率达到约 43%，说明模型快速掌握了部分基础特征。

```
C:\Users\chuxi>conda activate pytorch-cifar

(pytorch-cifar) C:\Users\chuxi>cd

(pytorch-cifar) C:\Users\chuxi>cd C:\Users\chuxi\Desktop\cifar10-project\pytorch-cifar-master

(pytorch-cifar) C:\Users\chuxi\Desktop\cifar10-project\pytorch-cifar-master>python main.py
==> Preparing data..
Files already downloaded and verified
Files already downloaded and verified
==> Building model..

Epoch: 0
==> Preparing data..
Files already downloaded and verified
Files already downloaded and verified
==> Building model..
==> Preparing data..
Files already downloaded and verified
Files already downloaded and verified
==> Building model..
[  1/391] Loss: 2.312 | Acc: 9.375% (12/128)
[ 11/391] Loss: 2.318 | Acc: 13.352% (188/1408)
[ 21/391] Loss: 2.270 | Acc: 15.885% (427/2688)
[ 31/391] Loss: 2.219 | Acc: 17.011% (675/3968)
[ 41/391] Loss: 2.178 | Acc: 18.502% (971/5248)
[ 51/391] Loss: 2.137 | Acc: 20.067% (1310/6528)
[ 61/391] Loss: 2.104 | Acc: 21.196% (1655/7808)
[ 71/391] Loss: 2.068 | Acc: 22.216% (2019/9088)
[ 81/391] Loss: 2.039 | Acc: 23.071% (2392/10368)
[ 91/391] Loss: 2.018 | Acc: 23.601% (2749/11648)
[101/391] Loss: 1.990 | Acc: 24.590% (3179/12928)
[111/391] Loss: 1.969 | Acc: 25.303% (3595/14208)
[121/391] Loss: 1.957 | Acc: 25.704% (3981/15488)
[131/391] Loss: 1.942 | Acc: 26.354% (4419/16768)
[141/391] Loss: 1.924 | Acc: 27.050% (4882/18048)
[151/391] Loss: 1.906 | Acc: 27.732% (5360/19328)
[161/391] Loss: 1.892 | Acc: 28.251% (5822/20608)
[171/391] Loss: 1.881 | Acc: 28.655% (6272/21888)
[181/391] Loss: 1.869 | Acc: 29.161% (6756/23168)
[191/391] Loss: 1.859 | Acc: 29.553% (7225/24448)
[201/391] Loss: 1.848 | Acc: 29.925% (7699/25728)
[211/391] Loss: 1.837 | Acc: 30.391% (8208/27008)
[221/391] Loss: 1.830 | Acc: 30.794% (8711/28288)
[231/391] Loss: 1.820 | Acc: 31.172% (9217/29568)
[241/391] Loss: 1.808 | Acc: 31.584% (9743/30848)
[251/391] Loss: 1.799 | Acc: 31.913% (10253/32128)
[261/391] Loss: 1.790 | Acc: 32.229% (10767/33408)
[271/391] Loss: 1.780 | Acc: 32.616% (11314/34688)
[281/391] Loss: 1.773 | Acc: 32.943% (11849/35968)
[291/391] Loss: 1.765 | Acc: 33.277% (12395/37248)
[301/391] Loss: 1.755 | Acc: 33.703% (12985/38528)
[311/391] Loss: 1.748 | Acc: 33.981% (13527/39808)
[321/391] Loss: 1.740 | Acc: 34.307% (14096/41088)
[331/391] Loss: 1.732 | Acc: 34.658% (14684/42368)
[341/391] Loss: 1.726 | Acc: 34.943% (15252/43648)
[351/391] Loss: 1.719 | Acc: 35.221% (15824/44928)
[361/391] Loss: 1.714 | Acc: 35.464% (16387/46208)
[371/391] Loss: 1.707 | Acc: 35.805% (17003/47488)
[381/391] Loss: 1.700 | Acc: 36.147% (17628/48768)
[391/391] Loss: 1.693 | Acc: 36.448% (18224/50000)
==> Preparing data..
Files already downloaded and verified
Files already downloaded and verified
==> Building model..
==> Preparing data..
Files already downloaded and verified
Files already downloaded and verified
==> Building model..
[  1/100] Loss: 1.560 | Acc: 44.000% (44/100)
[ 11/100] Loss: 1.557 | Acc: 43.182% (475/1100)
[ 21/100] Loss: 1.549 | Acc: 43.667% (917/2100)
[ 31/100] Loss: 1.559 | Acc: 43.710% (1355/3100)
[ 41/100] Loss: 1.553 | Acc: 43.976% (1803/4100)
[ 51/100] Loss: 1.552 | Acc: 43.922% (2240/5100)
[ 61/100] Loss: 1.547 | Acc: 44.131% (2692/6100)
[ 71/100] Loss: 1.558 | Acc: 43.915% (3118/7100)
[ 81/100] Loss: 1.552 | Acc: 43.691% (3539/8100)
[ 91/100] Loss: 1.562 | Acc: 43.253% (3936/9100)
[100/100] Loss: 1.562 | Acc: 43.190% (4319/10000)
Saving..
```

训练中期（第 20-30 轮）：模型快速学习，准确率显著提升。（本阶段暂未截图，但通过日志可确认损失持续下降、准确率上升）

训练后期（第 50+ 轮）：模型趋于收敛，准确率稳定在较高水平

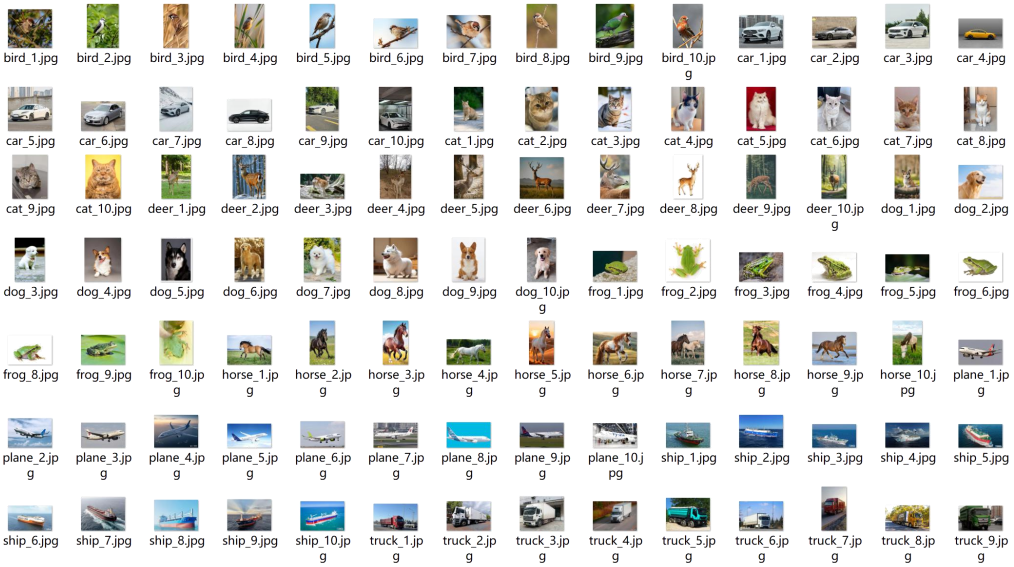
```
Epoch: 88
==> Preparing data..
Files already downloaded and verified
Files already downloaded and verified
==> Building model..
==> Resuming from checkpoint..
  checkpoint = torch.load('./checkpoint/ckpt.pth')
==> Preparing data..
Files already downloaded and verified
Files already downloaded and verified
==> Building model..
==> Resuming from checkpoint..
  checkpoint = torch.load('./checkpoint/ckpt.pth')
[  1/391] Loss: 0.291 | Acc: 88.281% (113/128)
[ 11/391] Loss: 0.331 | Acc: 88.423% (1245/1408)
[ 21/391] Loss: 0.346 | Acc: 88.356% (2375/2688)
[ 31/391] Loss: 0.357 | Acc: 87.702% (3480/3968)
[ 41/391] Loss: 0.354 | Acc: 88.129% (4625/5248)
[ 51/391] Loss: 0.350 | Acc: 88.097% (5751/6528)
[ 61/391] Loss: 0.343 | Acc: 88.384% (6901/7808)
[ 71/391] Loss: 0.340 | Acc: 88.534% (8046/9088)
[ 81/391] Loss: 0.340 | Acc: 88.551% (9181/10368)
[ 91/391] Loss: 0.338 | Acc: 88.573% (10317/11648)
[101/391] Loss: 0.337 | Acc: 88.567% (11450/12928)
[111/391] Loss: 0.339 | Acc: 88.514% (12576/14208)
[121/391] Loss: 0.339 | Acc: 88.481% (13704/15488)
[131/391] Loss: 0.335 | Acc: 88.663% (14867/16768)
[141/391] Loss: 0.335 | Acc: 88.625% (15995/18048)
[151/391] Loss: 0.334 | Acc: 88.643% (17133/19328)
[161/391] Loss: 0.333 | Acc: 88.660% (18271/20608)
[171/391] Loss: 0.333 | Acc: 88.660% (19406/21888)
[181/391] Loss: 0.334 | Acc: 88.635% (20535/23168)
[191/391] Loss: 0.335 | Acc: 88.600% (21661/24448)
[201/391] Loss: 0.335 | Acc: 88.592% (22793/25728)
[211/391] Loss: 0.334 | Acc: 88.633% (23938/27008)
[221/391] Loss: 0.334 | Acc: 88.645% (25076/28288)
[231/391] Loss: 0.336 | Acc: 88.552% (26183/29568)
[241/391] Loss: 0.336 | Acc: 88.537% (27312/30848)
[251/391] Loss: 0.337 | Acc: 88.527% (28442/32128)
[261/391] Loss: 0.337 | Acc: 88.521% (29573/33408)
[271/391] Loss: 0.337 | Acc: 88.526% (30708/34688)
[281/391] Loss: 0.337 | Acc: 88.518% (31838/35968)
[291/391] Loss: 0.336 | Acc: 88.507% (32967/37248)
[301/391] Loss: 0.336 | Acc: 88.525% (34107/38528)
[311/391] Loss: 0.335 | Acc: 88.563% (35255/39808)
[321/391] Loss: 0.334 | Acc: 88.593% (36401/41088)
[331/391] Loss: 0.333 | Acc: 88.616% (37545/42368)
[341/391] Loss: 0.335 | Acc: 88.584% (38665/43648)
[351/391] Loss: 0.336 | Acc: 88.562% (39789/44928)
[361/391] Loss: 0.336 | Acc: 88.519% (40903/46208)
[371/391] Loss: 0.336 | Acc: 88.521% (42037/47488)
[381/391] Loss: 0.335 | Acc: 88.558% (43188/48768)
[391/391] Loss: 0.335 | Acc: 88.580% (44290/50000)
==> Preparing data..
Files already downloaded and verified
Files already downloaded and verified
==> Building model..
==> Resuming from checkpoint..
==> Preparing data..
Files already downloaded and verified
Files already downloaded and verified
==> Building model..
==> Resuming from checkpoint..
  checkpoint = torch.load('./checkpoint/ckpt.pth')
[  1/100] Loss: 0.298 | Acc: 88.000% (88/100)
[ 11/100] Loss: 0.383 | Acc: 87.091% (958/1100)
[ 21/100] Loss: 0.393 | Acc: 87.381% (1835/2100)
[ 31/100] Loss: 0.407 | Acc: 86.645% (2686/3100)
[ 41/100] Loss: 0.414 | Acc: 86.390% (3542/4100)
[ 51/100] Loss: 0.407 | Acc: 86.725% (4423/5100)
[ 61/100] Loss: 0.406 | Acc: 86.607% (5283/6100)
[ 71/100] Loss: 0.404 | Acc: 86.648% (6152/7100)
[ 81/100] Loss: 0.400 | Acc: 86.605% (7015/8100)
[ 91/100] Loss: 0.403 | Acc: 86.527% (7874/9100)
[100/100] Loss: 0.400 | Acc: 86.630% (8663/10000)
Saving..
```

原项目完整训练需要 200 轮以达到 94%+的准确率，但考核主要验证流程的正确性，100 轮时模型已具备良好的识别能力（测试集准确率约 86.63%），因此提前停止训练，将时间用于 ResNet18 模型。

从训练初期的约 40% 到训练结束时的约 87%，模型准确率在整个训练过程中持续上升，损失值 Loss 逐步下降，准确率 Acc 波动中整体向上，模型学习过程正常。

3.1.4 外部图片测试结果

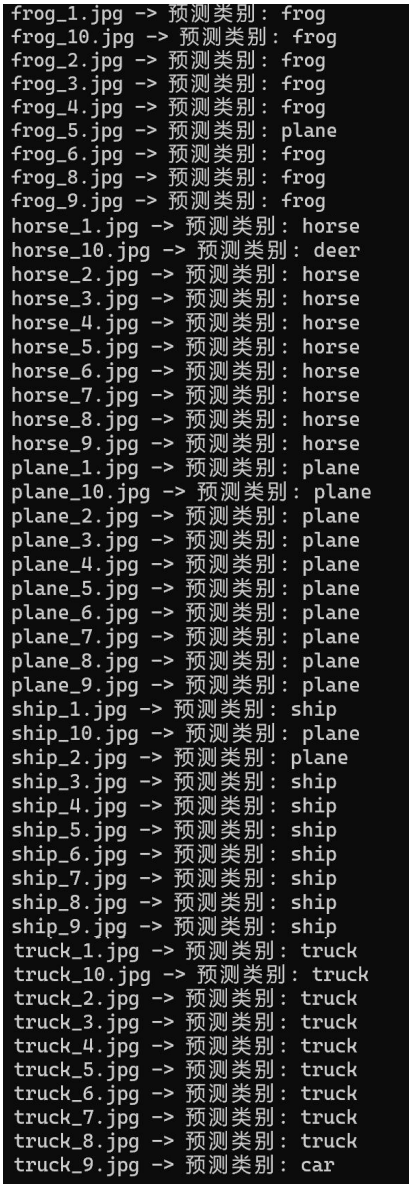
（1）从网络上下载了 10 个类别的图片，每类共 10 张，放入 test_images 文件夹中：



（2）编写测试脚本 test_simpledla.py，用于测试外部图片：

```
import torch
import torchvision.transforms as transforms
from PIL import Image
import os

# 从 dla_simple.py 导入 SimpleDLA
from models.dla_simple import SimpleDLA

device = 'cuda' if torch.cuda.is_available() else 'cpu'

# 加载模型
model = SimpleDLA().to(device)
checkpoint = torch.load('./checkpoint/ckpt.pth', map_location=device)
model.load_state_dict(checkpoint['net'])
model.eval()
print("SimpleDLA 模型加载成功！")

# 预处理
transform = transforms.Compose([
    transforms.Resize(32),
    transforms.CenterCrop(32),
    transforms.ToTensor(),
    transforms.Normalize((0.4914, 0.4822, 0.4465), (0.2023, 0.1994, 0.2010)),
])

classes = ('plane', 'car', 'bird', 'cat', 'deer',
           'dog', 'frog', 'horse', 'ship', 'truck')

# 测试图片文件夹
test_dir = './test_images'
if not os.path.exists(test_dir):
    os.makedirs(test_dir)
    print(f"已创建 {test_dir} 文件夹，请放入几张测试图片后重新运行")
    exit()

for img_name in os.listdir(test_dir):
    if img_name.endswith(('.jpg', '.png', '.jpeg')):
        img_path = os.path.join(test_dir, img_name)
        image = Image.open(img_path).convert('RGB')
        input_tensor = transform(image).unsqueeze(0)

        with torch.no_grad():
            output = model(input_tensor.to(device))
            _, predicted = output.max(1)

        print(f'{img_name} -> 预测类别: {classes[predicted.item()]}')
```

（3）外部图片测试结果如下：



将上图结果统计为如下表格：

类别	测试数量	正确数量	准确率
bird	10	6	60%
car	10	10	100%
cat	10	9	90%
deer	10	10	100%
dog	10	9	90%
frog	10	9	90%
horse	10	9	90%
plane	10	10	100%
ship	10	8	80%
truck	10	9	90%
总计	100	89	约 89%

3.1.5 错误案例分析

（1） 部分猫图片被识别为狗，部分狗图片被识别为猫。原因是 CIFAR-10 图片分辨率较低（32×32），猫狗在低分辨率下体型、轮廓等特征相似；

（2）轮船与飞机混淆：部分轮船被识别为飞机，可能是俯视角度下船体轮廓与飞机相似；

（3）鸟准确率偏低：测试图片中的鸟类背景复杂，如：树叶、草丛，与训练集中简单背景的鸟类存在差异。

3.2. 模型二：ResNet18（考核主要模型）

3.2.1 模型简介

ResNet18 是残差网络（Residual Network）的 18 层版本，通过残差连接解决了深层网络的梯度消失问题，是图像分类领域的经典模型。

3.2.2 训练过程

（1）模型修改：在 main.py 中将 net = SimpleDLA()改为 net = ResNet18()

```
# Model
print('==> Building model..')
# net = VGG('VGG19')
net = ResNet18()
# net = PreActResNet18()
# net = GoogLeNet()
# net = DenseNet121()
# net = ResNeXt29_2x64d()
# net = MobileNet()
# net = MobileNetV2()
# net = DPN92()
# net = ShuffleNetG2()
# net = SENet18()
# net = ShuffleNetV2(1)
# net = EfficientNetB0()
# net = RegNetX_200MF()
# net = SimpleDLA()
net = net.to(device)
if device == 'cuda':
    net = torch.nn.DataParallel(net)
    cudnn.benchmark = True
```

（2）训练轮数：200 轮

（3）训练耗时：约 30 小时

3.2.3 测试结果

根据最后一轮训练日志截图：在训练集上，模型表现接近完美，大部分 batch 的准确率达到 100%，最终训练集准确率接近 100%。在 CIFAR-10 测试集上，模型取得了 95.48% 的准确率，达到原项目报告的基准水平（ResNet18 原准确率为 93.02%），验证了 ResNet18 在 CIFAR-10 分类任务上的有效性。

```
Epoch: 199
==> Preparing data..
Files already downloaded and verified
Files already downloaded and verified
==> Building model..
==> Preparing data..
Files already downloaded and verified
Files already downloaded and verified
==> Building model..
[  1/391] Loss: 0.001 | Acc: 100.000% (128/128)
[ 11/391] Loss: 0.002 | Acc: 100.000% (1408/1408)
[ 21/391] Loss: 0.002 | Acc: 100.000% (2688/2688)
[ 31/391] Loss: 0.002 | Acc: 100.000% (3968/3968)
[ 41/391] Loss: 0.002 | Acc: 100.000% (5248/5248)
[ 51/391] Loss: 0.002 | Acc: 100.000% (6528/6528)
[ 61/391] Loss: 0.002 | Acc: 99.987% (7807/7808)
[ 71/391] Loss: 0.002 | Acc: 99.989% (9087/9088)
[ 81/391] Loss: 0.002 | Acc: 99.990% (10367/10368)
[ 91/391] Loss: 0.002 | Acc: 99.991% (11647/11648)
[101/391] Loss: 0.002 | Acc: 99.992% (12927/12928)
[111/391] Loss: 0.002 | Acc: 99.986% (14206/14208)
[121/391] Loss: 0.002 | Acc: 99.987% (15486/15488)
[131/391] Loss: 0.002 | Acc: 99.988% (16766/16768)
[141/391] Loss: 0.002 | Acc: 99.989% (18046/18048)
[151/391] Loss: 0.002 | Acc: 99.990% (19326/19328)
[161/391] Loss: 0.002 | Acc: 99.990% (20606/20608)
[171/391] Loss: 0.002 | Acc: 99.991% (21886/21888)
[181/391] Loss: 0.002 | Acc: 99.991% (23166/23168)
[191/391] Loss: 0.002 | Acc: 99.992% (24446/24448)
[201/391] Loss: 0.002 | Acc: 99.988% (25725/25728)
[211/391] Loss: 0.002 | Acc: 99.989% (27005/27008)
[221/391] Loss: 0.002 | Acc: 99.989% (28285/28288)
[231/391] Loss: 0.002 | Acc: 99.990% (29565/29568)
[241/391] Loss: 0.002 | Acc: 99.990% (30845/30848)
[251/391] Loss: 0.002 | Acc: 99.988% (32124/32128)
[261/391] Loss: 0.002 | Acc: 99.988% (33404/33408)
[271/391] Loss: 0.002 | Acc: 99.988% (34684/34688)
[281/391] Loss: 0.002 | Acc: 99.989% (35964/35968)
[291/391] Loss: 0.002 | Acc: 99.989% (37244/37248)
[301/391] Loss: 0.002 | Acc: 99.990% (38524/38528)
[311/391] Loss: 0.002 | Acc: 99.990% (39804/39808)
[321/391] Loss: 0.002 | Acc: 99.990% (41084/41088)
[331/391] Loss: 0.002 | Acc: 99.991% (42364/42368)
[341/391] Loss: 0.002 | Acc: 99.989% (43643/43648)
[351/391] Loss: 0.002 | Acc: 99.989% (44923/44928)
[361/391] Loss: 0.002 | Acc: 99.989% (46203/46208)
[371/391] Loss: 0.002 | Acc: 99.989% (47483/47488)
[381/391] Loss: 0.002 | Acc: 99.990% (48763/48768)
[391/391] Loss: 0.002 | Acc: 99.990% (49995/50000)
==> Preparing data..
Files already downloaded and verified
Files already downloaded and verified
==> Building model..
==> Preparing data..
Files already downloaded and verified
Files already downloaded and verified
==> Building model..
[  1/100] Loss: 0.138 | Acc: 95.000% (95/100)
[ 11/100] Loss: 0.170 | Acc: 94.909% (1044/1100)
[ 21/100] Loss: 0.179 | Acc: 95.286% (2001/2100)
[ 31/100] Loss: 0.182 | Acc: 95.194% (2951/3100)
[ 41/100] Loss: 0.186 | Acc: 95.171% (3902/4100)
[ 51/100] Loss: 0.184 | Acc: 95.275% (4859/5100)
[ 61/100] Loss: 0.182 | Acc: 95.344% (5816/6100)
[ 71/100] Loss: 0.180 | Acc: 95.394% (6773/7100)
[ 81/100] Loss: 0.178 | Acc: 95.481% (7734/8100)
[ 91/100] Loss: 0.176 | Acc: 95.473% (8688/9100)
[100/100] Loss: 0.176 | Acc: 95.480% (9548/10000)
```

3.2.4 外部图片测试结果

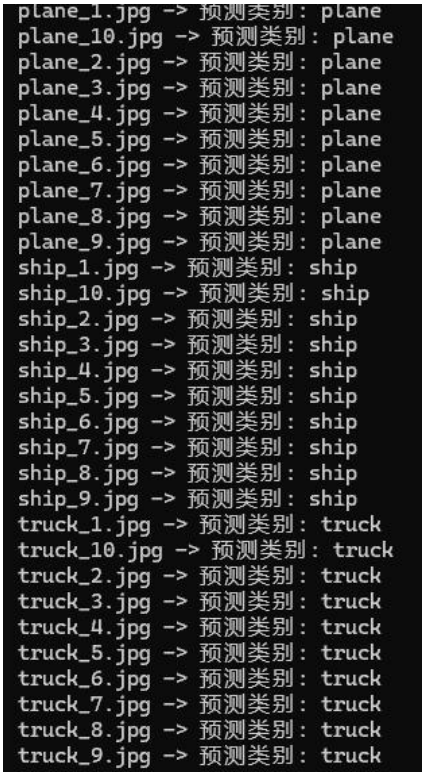
（1）使用与 SimpleDLA 相同的测试图片，并编写测试脚本 test_resnet18.py，用于测试外部图片:

```
import torch
import torchvision.transforms as transforms
from PIL import Image
import os

# 导入 ResNet18
from models.resnet import ResNet18

device = 'cuda' if torch.cuda.is_available() else 'cpu'

#使用 ResNet18 模型
model = ResNet18().to(device)
checkpoint = torch.load('./checkpoint/ckpt.pth', map_location=device)
model.load_state_dict(checkpoint['net'])
model.eval()
print("ResNet18 模型加载成功！")

# 预处理
transform = transforms.Compose([
    transforms.Resize(32),
    transforms.CenterCrop(32),
    transforms.ToTensor(),
    transforms.Normalize((0.4914, 0.4822, 0.4465), (0.2023, 0.1994, 0.2010)),
])

classes = ('plane', 'car', 'bird', 'cat', 'deer',
           'dog', 'frog', 'horse', 'ship', 'truck')

test_dir = './test_images'
if not os.path.exists(test_dir):
    os.makedirs(test_dir)
    print(f"已创建 {test_dir} 文件夹，请放入几张测试图片后重新运行")
    exit()

for img_name in os.listdir(test_dir):
    if img_name.endswith(('.jpg', '.png', '.jpeg')):
        img_path = os.path.join(test_dir, img_name)
        image = Image.open(img_path).convert('RGB')
        input_tensor = transform(image).unsqueeze(0)

        with torch.no_grad():
            output = model(input_tensor.to(device))
            _, predicted = output.max(1)

        print(f'{img_name} -> 预测类别: {classes[predicted.item()]}')
```

（2）外部图片测试结果如下

```
ResNet18 模型加载成功！
bird_1.jpg -> 预测类别: bird
bird_10.jpg -> 预测类别: bird
bird_2.jpg -> 预测类别: bird
bird_3.jpg -> 预测类别: bird
bird_4.jpg -> 预测类别: bird
bird_5.jpg -> 预测类别: bird
bird_6.jpg -> 预测类别: bird
bird_7.jpg -> 预测类别: bird
bird_8.jpg -> 预测类别: bird
bird_9.jpg -> 预测类别: bird
car_1.jpg -> 预测类别: car
car_10.jpg -> 预测类别: car
car_2.jpg -> 预测类别: car
car_3.jpg -> 预测类别: car
car_4.jpg -> 预测类别: car
car_5.jpg -> 预测类别: car
car_6.jpg -> 预测类别: car
car_7.jpg -> 预测类别: car
car_8.jpg -> 预测类别: car
car_9.jpg -> 预测类别: car
```

```
frog_1.jpg -> 预测类别: frog
frog_10.jpg -> 预测类别: frog
frog_2.jpg -> 预测类别: frog
frog_3.jpg -> 预测类别: frog
frog_4.jpg -> 预测类别: frog
frog_5.jpg -> 预测类别: frog
frog_6.jpg -> 预测类别: frog
frog_8.jpg -> 预测类别: frog
frog_9.jpg -> 预测类别: frog
horse_1.jpg -> 预测类别: horse
horse_10.jpg -> 预测类别: deer
horse_2.jpg -> 预测类别: horse
horse_3.jpg -> 预测类别: horse
horse_4.jpg -> 预测类别: horse
horse_5.jpg -> 预测类别: horse
horse_6.jpg -> 预测类别: horse
horse_7.jpg -> 预测类别: horse
horse_8.jpg -> 预测类别: horse
horse_9.jpg -> 预测类别: horse
```



将上图结果统计为如下表格：

类别	测试数量	正确数量	准确率
bird	10	10	100%
car	10	10	100%
cat	10	10	100%
deer	10	10	100%
dog	10	10	100%
frog	9	9	100%
horse	10	9	90%
plane	10	10	100%
ship	10	10	100%
truck	9	9	100%
总计	98	97	约 99%

3.2.5 错误案例分析

唯一错误： horse 被识别为 deer。

原因分析： 这张图片中的马可能采取了低头吃草或侧身姿态，身体轮廓与鹿相似；或者图片中马的角度、光线导致模型无法捕捉到典型的马的特征。在 CIFAR-10 的低分辨率下，马和鹿的区分本身具有一定难度。

四、总结

4.1 项目完成情况

（1） 完成轻量级模型 SimpleDLA 的完整训练与测试，并记录了训练过程中的准确率变化；

（2）按照考核要求，完成 ResNet18 模型的训练与测试，在官方测试集上达到 95.48% 的准确率，超过原项目的基准表现（93.02%）；

（3）自行收集并测试了 10 个类别共 100 张外部图片，分别使用 SimpleDLA 和 ResNet18 进行推理，验证了模型在实际图片上的泛化能力。

4.2 学习收获

通过本次项目，我在多个方面获得了实践经验与认知提升：

4.2.1 技术能力

（1）掌握了深度学习项目的完整流程：从环境配置中的 Conda 虚拟环境和依赖安装，到数据处理中的下载、预处理与数据增强，再到模型训练涉及的损失函数、优化器和学习率调度，最后到模型评估的测试集准确率计算及外部图片推理，由此形成清晰的工程化思维，能够独立复现开源项目并完成自定义改进。

（2）熟悉了 PyTorch 框架的基本使用：能够熟练使用 torch.utils.data.DataLoader 加载数据，定义和修改模型，编写训练循环，使用 torch.save / torch.load 保存和恢复模型，并利用 --resume 实现断点续训。

（3）理解了模型训练过程中的关键指标：观察损失函数值 Loss 的下降趋势和准确率 Acc 的上升趋势，能够判断模型是否正常学习；理解为何验证集/测试集准确率比训练集准确率更能反映模型真实能力；学会在准确率创新高时保存模型，防止过拟合。

（4）掌握了实验记录与报告撰写的规范：包括截图保留关键输出、整理数据表格、分析错误案例、对比不同模型差异等，为今后撰写更规范的科研报告打下基础。

4.2.2 科研素养

（1）对科研流程有了初步认识：从选题、方案设计、实验执行、结果分析到文档撰写，体验了一个完整的研究闭环。理解了复现代码不仅是跑通，更要理解设计思想、记录细节、分析结果。

（2）培养了独立学习和查阅资料的能力：教程中未解决的问题，主动通过搜索引擎、GitHub Issues、技术社区等渠道查找解决方案，并学会根据报错信息判断问题类型。

4.3 未来改进方向

尽管本次项目已基本完成考核要求，但仍有进一步提升的空间：

硬件加速：后续可借助云 GPU 加速训练，尝试更深层网络或更大数据集；

模型探索：尝试 ResNet50、DenseNet、EfficientNet 等更先进的架构，对比它们在 CIFAR-10 上的性能与训练速度；

超参数调优：系统调整学习率、批量大小、权重衰减等参数，观察对收敛速度与最终准确率的影响；

数据增强：引入更丰富的数据增强策略（如随机旋转、色彩抖动、CutOut 等），进一步提升模型的泛化能力。

部署尝试：将训练好的模型导出为 ONNX 或 TorchScript，并在简单应用中测试实时推理。